Post-Improvement of Capacitated Vehicle Routing Problem Using Weighted Partial Max-SAT

Technical Documentation

Based on: Ferreira & Arruda (2024)

December 19, 2025

Contents

1 Introduction

1.1 Problem Overview

The Capacitated Vehicle Routing Problem (CVRP) is a fundamental combinatorial optimization problem in logistics and transportation. Given a fleet of vehicles with limited capacity stationed at a depot, the goal is to determine optimal routes to serve a set of customers with known demands while minimizing total travel distance.

1.2 Approach Summary

This document describes a **post-improvement approach** that combines:

1. **Clarke-Wright Savings Algorithm**: Constructs an initial feasible solution
2. **2-Opt Local Search**: Improves each route individually
3. **K-Nearest Neighbors**: Expands the search space
4. **Max-SAT Solver (clasp)**: Performs global optimization

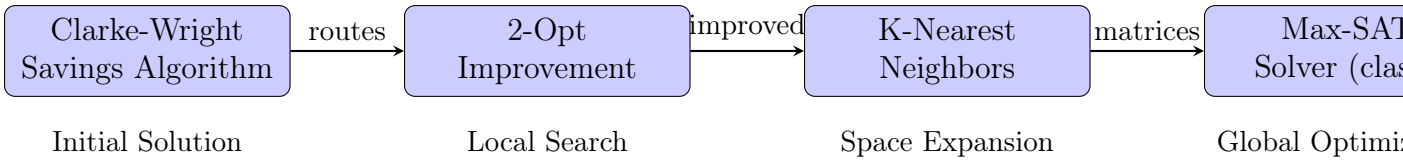


Figure 1: Pipeline of the CVRP Post-Improvement Approach

2 Problem Formulation

2.1 Formal Definition

Definition 2.1 (Capacitated Vehicle Routing Problem)**.** Let $G = (V, E)$ be an undirected graph where:

- $V = \{1, 2, \ldots, n\}$ is the set of vertices
- Node 1 represents the **depot**
- Nodes $\{2, \ldots, n\}$ represent **customer locations**
- Each customer i has demand $q_i \geq 0$
- Each edge $(i, j) \in E$ has cost d_{ij} (distance)
- Fleet of m vehicles, each with capacity Q

Objective: Partition customers into m routes, each starting and ending at depot, such that:

- Total demand per route $\leq Q$
- Total travel distance is minimized

2.2 Mathematical Model

Objective Function:

$$\min \sum_{i=1}^{n} \sum_{j=1}^{n} \sum_{v=1}^{m} d_{ij} \cdot w_{ijv} \tag{1}$$

where $w_{ijv} = 1$ if vehicle v travels directly from customer i to customer j.

3 Phase 1: Clarke-Wright Savings Algorithm

3.1 Algorithm Description

The Clarke-Wright Savings Algorithm is a greedy heuristic that builds routes by merging customer pairs based on "savings" values.

Definition 3.1 (Savings Value)**.** For customers $i, j \neq 1$ (depot), the savings from merging routes serving i and j is:

$$s_{ij} = d_{1i} + d_{1j} - d_{ij} \tag{2}$$

Interpretation: Instead of two separate round-trips (depot $\to i \to$ depot and depot $\to j \to$ depot), we combine into one route (depot $\to i \to j \to$ depot), saving distance s_{ij}.

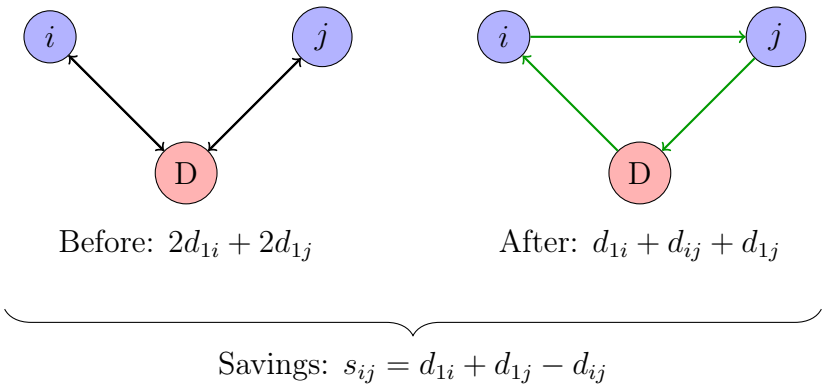


Figure 2: Illustration of Savings Concept

3.2 Algorithm Steps

Algorithm 1 Clarke-Wright Savings Algorithm (Modified for CVRP)

Require: Graph G, demands q_i, capacity Q, number of vehicles m
Ensure: Set of routes $\mathcal{R}$

Step 1: Compute Savings
for $i = 2$ to n **do**
    for $j = i + 1$ to n **do**
        $s_{ij} \leftarrow d_{1i} + d_{1j} - d_{ij}$
    end for
end for
Sort savings in descending order
Step 2: Initialize Routes
for each customer $i \in \{2, \ldots, n\}$ **do**
    Create route $R_i = [i]$ with demand q_i
end for
Step 3: Merge Routes
for each (s_{ij}, i, j) in sorted savings list **do**
    if i and j are in different routes R_i and R_j **then**
        if i is at end of R_i AND j is at start of R_j **then**
            if demand(R_i) + demand$(R_j) \leq Q$ **then**
                Merge: $R_i \leftarrow R_i \cup R_j$
                Delete R_j
            end if
        end if
    end if
end for
Step 4: Reduce Routes (if $|\mathcal{R}| > m$)
while $|\mathcal{R}| > m$ **do**
    Try to redistribute customers from smallest route
end while
return $\mathcal{R}$

3.3 Implementation Details

Listing 1: Clarke-Wright Savings Computation

```
def load_savings(self):
    '''Compute savings for all customer pairs'''
    for i in range(1, self.cvrp.dimension):
        for j in range(i + 1, self.cvrp.dimension):
            # s_ij = d_1i + d_1j - d_ij
            saving = (self.cvrp.distances[0, i] +
                      self.cvrp.distances[0, j] -
                      self.cvrp.distances[i, j])
            self.savings.append((saving, i, j))

    # Sort in descending order
    self.savings.sort(key=lambda x: x[0], reverse=True)
```

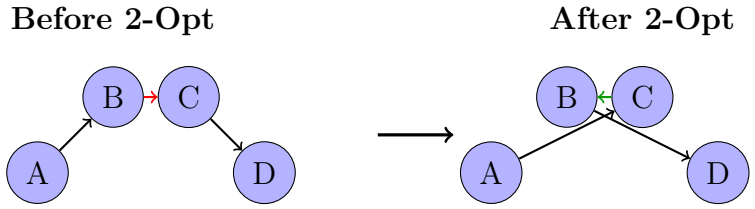
Listing 2: Route Merging with Capacity Constraint

```
def combine_routes(self):
    '''Merge routes based on savings'''
    for saving, i, j in self.savings:
        if i not in self.routes or j not in self.routes:
            continue
        if self.routes[i] == self.routes[j]:
            continue

        # Ensure i is at end and j is at start
        if self.routes[i][0] == i:
            self.routes[i] = self.routes[i].reversed()
        if self.routes[j][-1] == j:
            self.routes[j] = self.routes[j].reversed()

        if self.routes[i][-1] != i or self.routes[j][0] != j:
            continue

        # Check capacity constraint
        if self.routes[i].demand + self.routes[j].demand > self.
           cvrp.capacity:
            continue

        # Merge routes
        self.routes[i] += self.routes[j]
        del self.routes[j]
```

4 Phase 2: 2-Opt Local Search

4.1 Algorithm Description

The 2-Opt algorithm improves a route by iteratively removing two edges and reconnecting the tour in a different way.



Segment [B, C] is reversed

Figure 3: 2-Opt Move: Reverse segment between positions i and j

4.2 Algorithm

Algorithm 2 2-Opt Improvement

Require: Route $R = [v_1, v_2, \ldots, v_k]$
Ensure: Improved route R^*

$R^* \leftarrow R$
repeat
  improved $\leftarrow$ False
  for $i = 0$ to $|R| - 2$ **do**
    for $j = i + 1$ to $|R| - 1$ **do**
      $R' \leftarrow$ Reverse segment $[i, j]$ in R^*
      if cost(R') < cost(R^*) **then**
        $R^* \leftarrow R'$
        improved $\leftarrow$ True
      end if
    end for
  end for
until NOT improved
return R^*

4.3 Implementation

Listing 3: 2-Opt Implementation

```
def improve_routes(self):
    '''Apply 2-opt to each route'''
    for idx in self.routes:
        best_route = self.routes[idx]
        route = best_route

        while True:
            improved = False
            for i in range(len(route) - 1):
                for j in range(i + 1, len(route)):
                    # Reverse segment [i, j+1]
                    new_route = route.reversed(i, j + 1)
                    if new_route.cost < best_route.cost:
                        best_route = new_route
                        improved = True

            route = best_route
            if not improved:
                break

        self.routes[idx] = route
```

5 Phase 3: K-Nearest Neighbors Expansion

5.1 Motivation

After 2-Opt, the solution is locally optimal but may be far from the global optimum. To enable the SAT solver to explore beyond the current solution, we expand the search space by adding K-nearest neighbors for each customer.

5.2 Algorithm

Algorithm 3 K-Nearest Neighbors Selection

Require: Instance I, routes $\mathcal{R}$, parameter K
Ensure: Distance matrices $\{M_v\}$ for each vehicle v

Compute Minimum Spanning Tree (MST) of full graph
for each route R_v in $\mathcal{R}$ **do**
    Initialize matrix M_v with -1 (invalid edges)
    ▷ Add edges from current route
    for each consecutive pair (i, j) in R_v **do**
        $M_v[i, j] \leftarrow d_{ij}$
        $M_v[j, i] \leftarrow d_{ji}$
    end for
    ▷ Add K-nearest neighbors for each customer
    for each customer c in R_v **do**
        neighbors $\leftarrow$ K-nearest from MST
        if |neighbors| $< K$ **then**
            Fill remaining from distance matrix
        end if
        for each neighbor n in neighbors **do**
            $M_v[c, n] \leftarrow d_{cn}$
            $M_v[n, c] \leftarrow d_{nc}$
        end for
    end for
end for
return $\{M_v\}$

5.3 Implementation

Listing 4: K-Nearest Neighbors using MST

```
def load_mst(self):
    '''Build Minimum Spanning Tree'''
    graph = Graph()
    for i in range(self.cvrp.dimension):
        for j in range(self.cvrp.dimension):
            graph.add_edge(i, j, weight=self.cvrp.distances[i, j])
    self.mst = minimum_spanning_tree(graph)

def nearest_neighbors_mst(self, customer: int) -> list[int]:
```

```
    '''Get K nearest neighbors from MST'''
    neighbors = list(self.mst.neighbors(customer))
    weights = [self.mst.get_edge_data(customer, n)['weight']
               for n in neighbors]
    sorted_neighbors = [n for _, n in sorted(zip(weights, neighbors
       ))]
    return sorted_neighbors[:self.neighbor_number]

def load_matrices(self):
    '''Create distance matrices with K-neighbors'''
    for idx in self.routes:
        route = self.routes[idx]
        matrix = np.full((self.cvrp.dimension, self.cvrp.dimension)
           ,
                         -1, dtype=int)

        # Add current route edges
        for i in range(self.cvrp.dimension):
            matrix[i, i] = 0

        matrix[0, route[0]] = self.cvrp.distances[0, route[0]]
        for i in range(len(route) - 1):
            matrix[route[i], route[i+1]] = self.cvrp.distances[
               route[i], route[i+1]]
        matrix[0, route[-1]] = self.cvrp.distances[0, route[-1]]

        # Add K-nearest neighbors
        for customer in route:
            for neighbor in self.nearest_neighbors(customer):
                matrix[customer, neighbor] = self.cvrp.distances[
                   customer, neighbor]

        self.matrices[idx] = matrix
```

6 Phase 4: Max-SAT Logical Modeling

6.1 Boolean Variables

The CVRP is encoded using three types of Boolean variables:

Variable	Meaning
w_{ijv}	Vehicle v travels directly from customer i to customer j
t_{iv}	Customer i is assigned to vehicle v
u_{iv}	Position of customer i in route of vehicle v (for subtour elimination)

Table 1: Boolean Variables for CVRP Encoding

Binary Encoding of Position Variables:

To encode the integer position u_{iv} in binary form:

$$u_{iv} = \sum_{b=0}^{\lceil \log_2(n-1) \rceil - 1} 2^b \cdot u_{ibv} \tag{3}$$

where u_{ibv} are Boolean variables representing bits.

6.2 Constraint Formulation

6.2.1 Depot Constraints

Each vehicle must leave and return to the depot exactly once:

$$\sum_{j=2}^{n} w_{1jv} = 1, \quad \forall v \in \{1, \ldots, m\} \qquad \text{(C1: Leave depot)}$$

$$\sum_{i=2}^{n} w_{i1v} = 1, \quad \forall v \in \{1, \ldots, m\} \qquad \text{(C2: Return to depot)}$$

6.2.2 Customer Visit Constraints

Each customer must be visited exactly once by exactly one vehicle:

$$\sum_{j=1}^{n} \sum_{v=1}^{m} w_{ijv} = 1, \quad \forall i \in \{2, \ldots, n\}, i \neq j \qquad \text{(C3: One exit)}$$

$$\sum_{i=1}^{n} \sum_{v=1}^{m} w_{ijv} = 1, \quad \forall j \in \{2, \ldots, n\}, i \neq j \qquad \text{(C4: One entry)}$$

6.2.3 No Revisit Constraint

A vehicle cannot visit the same customer twice:

$$\neg w_{ijv} \vee \neg w_{jiv}, \quad \forall i < j, \forall v \qquad \text{(C5: No back-and-forth)}$$

In linear form: $-w_{ijv} - w_{jiv} \geq -1$

6.2.4 Vehicle-Customer Assignment Constraints

If vehicle v travels from i to j, then both customers are assigned to v:

$$w_{ijv} \Rightarrow t_{iv}, \quad \forall i, j \in \{2, \ldots, n\}, i \neq j, \forall v \tag{C6}$$

$$w_{ijv} \Rightarrow t_{jv}, \quad \forall i, j \in \{2, \ldots, n\}, i \neq j, \forall v \tag{C7}$$

In linear form: $-w_{ijv} + t_{iv} \geq 0$ and $-w_{ijv} + t_{jv} \geq 0$

6.2.5 Exclusive Assignment Constraint

Each customer is assigned to exactly one vehicle:

$$\neg t_{iv} \vee \neg t_{il}, \quad \forall i \in \{2, \ldots, n\}, \forall v \neq l \tag{C8}$$

6.2.6 Depot-Customer Link Constraints

$$w_{1jv} \Rightarrow t_{jv}, \quad \forall j \in \{2, \ldots, n\}, \forall v \qquad \text{(C9: After leaving depot)}$$

$$w_{i1v} \Rightarrow t_{iv}, \quad \forall i \in \{2, \ldots, n\}, \forall v \qquad \text{(C10: Before returning)}$$

6.2.7 Subtour Elimination (MTZ Formulation)

Miller-Tucker-Zemlin constraints prevent subtours:

$$u_{iv} - u_{jv} + (n-1) \cdot w_{ijv} \leq n - 2, \quad \forall i, j \in \{2, \ldots, n\}, i \neq j, \forall v \qquad \text{(C11)}$$

Interpretation: If $w_{ijv} = 1$ (vehicle v goes from i to j), then $u_{jv} \geq u_{iv} + 1$, ensuring a linear ordering of visited customers.

6.2.8 Capacity Constraint

$$\sum_{i=1}^{n} q_i \cdot t_{iv} \leq Q, \quad \forall v \in \{1, \ldots, m\} \qquad \text{(C12)}$$

6.3 Objective Function (Soft Clauses)

The objective is to minimize total travel distance:

$$\min \sum_{i=1}^{n} \sum_{j=1}^{n} \sum_{v=1}^{m} d_{ij} \cdot w_{ijv} \qquad (4)$$

In Max-SAT, this is encoded as soft clauses:

- For each w_{ijv}, add soft clause $(\neg w_{ijv}, d_{ij})$
- Satisfying the clause (not using edge) adds d_{ij} to the objective
- Falsifying the clause (using edge) incurs penalty d_{ij}

6.4 Search Space Restriction

Edges not in the K-neighbors matrix are forced to be false:

$$w_{ijv} = 0, \quad \forall (i, j) \text{ where } M_v[i, j] = -1 \qquad (5)$$

This dramatically reduces the search space.

6.5 Implementation

Listing 5: Loading Model Constraints

```
def load_model(self):
    '''Encode CVRP as Max-SAT'''
    bytes_size = ceil(log2(self.cvrp.dimension - 1))

    # C1: Each vehicle leaves depot by one customer
```

```
    for v in range(len(self.matrices)):
        w_0_j_v = [self.get(f'w_0_{j}_{v}')
                   for j in range(1, self.cvrp.dimension)]
        self.add_constraint_eq(None, w_0_j_v, 1)

    # C2: Each vehicle enters depot by one customer
    for v in range(len(self.matrices)):
        w_i_0_v = [self.get(f'w_{i}_0_{v}')
                   for i in range(1, self.cvrp.dimension)]
        self.add_constraint_eq(None, w_i_0_v, 1)

    # C3: Customer leaves to exactly one customer by one vehicle
    for i in range(1, self.cvrp.dimension):
        w_i_j_v = []
        for v in range(len(self.matrices)):
            w_i_j_v += [self.get(f'w_{i}_{j}_{v}')
                        for j in range(self.cvrp.dimension) if i !=
                            j]
        self.add_constraint_eq(None, w_i_j_v, 1)

    # C4: Customer entered by exactly one customer by one vehicle
    for j in range(1, self.cvrp.dimension):
        w_i_j_v = []
        for v in range(len(self.matrices)):
            w_i_j_v += [self.get(f'w_{i}_{j}_{v}')
                        for i in range(self.cvrp.dimension) if i !=
                            j]
        self.add_constraint_eq(None, w_i_j_v, 1)

    # C5: No back-and-forth
    for i in range(1, self.cvrp.dimension):
        for j in range(i + 1, self.cvrp.dimension):
            for v in range(len(self.matrices)):
                w_i_j_v = self.get(f'w_{i}_{j}_{v}')
                w_j_i_v = self.get(f'w_{j}_{i}_{v}')
                self.add_constraint_geq(None, [-w_i_j_v, -w_j_i_v],
                    1)

    # ... (C6-C10 similar pattern)

    # C11: Subtour Elimination (MTZ)
    exp = [2**b for b in range(bytes_size)]
    neg_exp = [-item for item in exp]
    u_factors = neg_exp + exp + [-self.cvrp.dimension + 1]
    u_value = -self.cvrp.dimension + 2

    for v in range(len(self.matrices)):
        for i in range(1, self.cvrp.dimension):
            for j in range(1, self.cvrp.dimension):
                if i == j:
                    continue
```

```
                u_i_v = [self.get(f'u_{i}_{b}_{v}') for b in range(
                    bytes_size)]
                u_j_v = [self.get(f'u_{j}_{b}_{v}') for b in range(
                    bytes_size)]
                w_i_j_v = self.get(f'w_{i}_{j}_{v}')
                u_clause = u_i_v + u_j_v + [w_i_j_v]
                self.add_constraint_geq(u_factors, u_clause,
                    u_value)

    # C12: Capacity constraint
    neg_demands = [-demand for demand in self.cvrp.demands]
    for v in range(len(self.matrices)):
        t_i_v = [self.get(f't_{i}_{v}') for i in range(self.cvrp.
            dimension)]
        self.add_constraint_geq(neg_demands, t_i_v, -self.cvrp.
            capacity)

    # Restrict search space (invalid edges = 0)
    w_invalid = []
    for v in range(len(self.matrices)):
        for i in range(self.cvrp.dimension):
            for j in range(self.cvrp.dimension):
                if i != j and self.matrices[v][i, j] == -1:
                    w_invalid.append(self.get(f'w_{i}_{j}_{v}'))
    self.add_constraint_eq(None, w_invalid, 0)

    # Objective: minimize total distance
    for v in range(len(self.matrices)):
        for i in range(self.cvrp.dimension):
            for j in range(self.cvrp.dimension):
                if i != j:
                    w_i_j_v = self.get(f'w_{i}_{j}_{v}')
                    self.add_objective(self.cvrp.distances[i, j],
                        w_i_j_v)
```

7 Complete Pipeline

Algorithm 4 Complete CVRP Post-Improvement Pipeline

Require: Instance file, number of vehicles m, number of neighbors K
Ensure: Optimized routes and total cost

Phase 1: Load Instance
$I \leftarrow$ LoadInstance(instance_file)
Phase 2: Clarke-Wright
$\mathcal{R} \leftarrow$ ClarkeWright(I, m)
$cost_{CW} \leftarrow \sum_{R \in \mathcal{R}} \text{cost}(R)$
Phase 3: 2-Opt Improvement
for each route R in $\mathcal{R}$ **do**
    $R \leftarrow$ TwoOpt(R)
end for
$cost_{2opt} \leftarrow \sum_{R \in \mathcal{R}} \text{cost}(R)$
Phase 4: K-Nearest Neighbors
$\{M_v\} \leftarrow$ KNeighbors(I, K, $\mathcal{R}$)
Phase 5: Max-SAT Solving
solver $\leftarrow$ CreateSolver(I, $\{M_v\}$)
solver.LoadModel()
solver.Solve()
$cost_{SAT} \leftarrow$ solver.optimum
routes $\leftarrow$ solver.DecodeRoutes()
Output Results
Print "CW+2Opt cost: $cost_{2opt}$"
Print "SAT cost: $cost_{SAT}$"
Print "Improvement: $\frac{cost_{2opt} - cost_{SAT}}{cost_{2opt}} \times 100\%$"
return routes, $cost_{SAT}$

Listing 6: Main Execution Script

```
from classes import Instance, ClarkeWright, TwoOpt, KNeighbors,
    Solver

if __name__ == '__main__':
    # Load instance
    cvrp = Instance('instances/A-n33-k6.vrp').load()

    # Phase 1-2: Clarke-Wright + 2-Opt
    cw_time, routes = ClarkeWright.run(cvrp, vehicle_number=6)
    to_time, routes = TwoOpt.run(routes)

    cw_cost = sum(route.cost for route in routes.values())
    print(f'CW + 2-Opt cost: {cw_cost}')

    # Phase 3: K-Nearest Neighbors
    _, matrices = KNeighbors.run(cvrp, neighbor_number=5, routes=
        routes)

```

```
    # Phase 4: SAT Solver
    solver_time, solver_cost, solver_routes = Solver.run(cvrp,
        matrices)

    print(f'Solver cost: {solver_cost}')
    print(f'Improvement: {(cw_cost - solver_cost) / cw_cost *
        100:.2f}%')
```

8 Experimental Results

8.1 Benchmark Instances

Instance	Optimal	CW+2Opt	K=5 Cost	Time (s)	Improvement
A-n33-k6	742	995	916	0.538	7.9%
A-n37-k5	669	1160	1020	0.979	12.1%
E-n31-k7	379	647	594	0.206	8.2%
F-n45-k4	724	1292	1194	7.246	7.6%
P-n45-k5	510	718	659	34.251	8.2%
P-n101-k4	681	1162	1034	90.314	11.0%

Table 2: Benchmark Results with K=5 Nearest Neighbors

8.2 Effect of K Parameter

- **K=3**: Fast computation, limited improvement
- **K=4**: Moderate balance
- **K=5**: Best solution quality, longer computation time

Larger K expands the search space, allowing the SAT solver to find better solutions at the cost of increased computational time.

9 Comparison with RTSS Approach

Aspect	CVRP (This Paper)	RTSS (Zha et al.)
Problem Type	Static routing	Real-time dynamic allocation
Initial Solution	Clarke-Wright + 2-Opt	None (pure SAT)
SAT Approach	Post-improvement only	Full encoding from scratch
Search Space	Limited by K-neighbors	Full space
Incremental	No	Yes (lazy constraints)
Subtour Elimination	MTZ formulation	Reachability variables
Time Constraints	Capacity only	Deadline + Capacity
Solver	clasp	Modified QMaxSAT

Table 3: Comparison of CVRP and RTSS Max-SAT Approaches

10 Conclusion

The post-improvement approach for CVRP using Max-SAT demonstrates:

1. **Effective Hybrid Strategy**: Combining heuristics with exact methods
2. **Search Space Reduction**: K-neighbors limits complexity while enabling improvement
3. **Practical Results**: 7-12% improvement over heuristic solutions
4. **Reasonable Time**: Solutions found within 100 seconds for most instances

Key Insights:

- Heuristics provide good starting points but may miss global optima
- Max-SAT can explore structured search spaces efficiently
- The K-neighbors parameter controls the trade-off between quality and time

References

1. Ferreira, P.H., Arruda, A.M. (2024). Post-improving the Capacitated Vehicle Routing Problem Using a Max-SAT Solver. *Brazilian Workshop of Logic*.
2. Clarke, G., Wright, J.W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*, 12(4):568-581.
3. Croes, G.A. (1958). A method for solving traveling-salesman problems. *Operations Research*, 6(6):791-812.
4. Gebser, M., et al. (2007). clasp: A conflict-driven answer set solver. *LPNMR*, pages 260-265.
5. Vidal, T. (2022). Hybrid genetic search for the CVRP: Open-source implementation and swap* neighborhood. *Computers & Operations Research*, 140:105643.